TOBB Ekonomi ve Teknoloji Üniversitesi

Elektrik-Elektronik Mühendisliği Bölümü

ELE 568 - Ağ Bilimi

Proje: GitHub Sosyal Ağının Yapısal Analizi

(musae-GitHub Veri Seti Üzerinde Ağ Bilimi Analizi)

Bahar, 2026

Eda İnan

221201076

İÇİNDEKİLER

1. GİRİŞ 1
2. VERİ SETİ VE AĞ TANIMI 2
2.1. Veri Kaynağı 2
2.2. Ağ Tanımı 2
2.3. Veri Formatı ve Ön İşleme 2
2.4. Temel İstatistikler 3
3. YÖNTEM VE KULLANILAN ARAÇLAR 4
4. DERECE DAĞILIMI ANALİZİ 5
4.1. Yöntem 5
4.2. Sonuçlar 7
4.3. Karşılaştırma 8
4.4. Yorum 9
5. MESAFE VE KÜÇÜK DÜNYA ANALİZİ 10
5.1. Yöntem 10
5.2. Sonuçlar 11
5.3. Karşılaştırma 12
5.4. Yorum 13
6. DERECE KORELASYONLARI 13
6.1. Yöntem 13
6.2. Sonuçlar 17
6.3. Yorum 18
6.4. Ek Analiz 19
7. GÜRBÜZLÜK ANALİZİ 19
7.1. Yöntem 19
7.2. Sonuçlar 21
7.3. Kritik Eşik Analizi 23
7.4. Yorum 23
8. TOPLULUK ANALİZİ 24
8.1. Yöntem 24
8.2. Sonuçlar 25
8.3. Küme Analizi 26
8.4. Ek Analiz 26
8.5. Yorum 27
9. GENEL DEĞERLENDİRME VE SONUÇ 27
9.1. Tüm Bulguların Özeti 27
9.2. Ağın Karakteristik Özellikleri 28
9.3. Ağ Modeli Karşılaştırması 29
9.4. Ağ Görselleştirmesi 30
9.5. Gerçek Dünya Yorumu 33
10. KAYNAKLAR 34

1. GİRİŞ

Bu rapor, GitHub üzerindeki yazılımcılar arasındaki sosyal etkileşim ağının yapısal özelliklerini, kullanıcıların birbirlerini karşılıklı takip etme (mutual follower) ilişkileri üzerinden incelemektedir. Oluşturulan ağ modeli, Barabási'nin "Network Science" prensipleri çerçevesinde **[1]** analiz edilmiştir. Modern yazılım dünyasında işbirliği ağlarının dinamiklerini anlamak, bilginin yayılmasını ve sistemin açık kaynak ekosistemindeki dayanıklılığını ölçmek açısından önemlidir.

GitHub platformu, geliştiriciler arasındaki etkileşimin açık biçimde gözlemlenebildiği ve gerçek dünya sosyal ağlarının özelliklerini yansıtan önemli bir örnektir. Bu nedenle, bu ağın analizi yalnızca teorik kavramların doğrulanması değil aynı zamanda açık kaynak ekosisteminin yapısal dinamiklerinin anlaşılması açısından da önemlidir.

Çalışmanın temel amacı; gerçek dünya ağlarının temel karakteristik özellikleri olan "*ölçekten bağımsızlık"* (scale-free), "*küçük dünya*" (small-world) ve "*topluluk yapısı*" (community structure) özelliklerini gözleme dayalı olarak doğrulamaktır.

Bu raporda temel olarak:

- GitHub ağı bir güç yasasına (power-law) mı uymakta, yoksa rastgele bir yapıya mı sahiptir?
- Ağ, stratejik düğümlerin kaybına karşı ne kadar gürbüzdür?
- Yazılımcılar belirli teknolojik veya sosyal kümeler (topluluklar) oluşturur mu?
- GitHub ağı hangi ağ üretim modeli (Barabási-Albert, Erdős-Rényi) ile en iyi açıklanabilir?

gibi sorulara yanıt aranmıştır.

2. VERİ SETİ VE AĞ TANIMI

2.1. Veri Kaynağı

Bu çalışmada kullanılan veri seti, Stanford Network Analysis Project (SNAP) kütüphanesinden alınan GitHub Social Network verisidir **[2]**. Veri setine aşağıdaki bağlantı üzerinden erişilmiştir:

https://snap.stanford.edu/data/github-social.html

Veri, 2019 yılındaki kamuya açık kullanıcı profil etkileşimlerini içermektedir. Veri seti, SNAP sayfasında yer alan git_web_ml.zip dosyası indirilerek elde edilmiştir. ZIP arşivi ayıklandıktan sonra analiz için yalnızca musae_git_edges.csv dosyası kullanılmıştır.

2.2. Ağ Tanımı

Düğümler (Nodes): GitHub platformundaki bireysel kullanıcıları temsil eder.

Kenarlar (Edges): İki düğüm arasında bir kenar bulunması, söz konusu iki kullanıcının birbirini karşılıklı olarak takip ettiğini (mutual follower) göstermektedir. Tek yönlü takip ilişkileri veri setine dahil edilmemiştir.

Ağ Tipi: Yönsüz ve ağırlıksız ve tek parçalı (unipartite) bir sosyal ağdır. Karşılıklı takip koşulu nedeniyle tüm kenarlar simetriktir. Ağın yoğunluğu düşüktür, bu da gerçek dünya sosyal ağlarının seyrek (sparse) yapısıyla uyumludur.

2.3. Veri Formatı ve Ön İşleme

Veri, her satırın bir kenarı temsil ettiği kenar listesi formatında CSV dosyası olarak sunulmaktadır. Dosyanın ilk satırı sütun başlıklarını (id_1, id_2) içermektedir. Düğümler sıfırdan başlayan tamsayı indeksleri ile etiketlenmiştir.

Veri seti doğrudan analiz edilebilir formatta olduğundan ek bir veri temizleme yapılmasına ihtiyaç duyulmamıştır. Kenar listesi doğrudan NetworkX kütüphanesine **[7]** aktarılmış, nx.Graph() sınıfı kullanılarak yönsüz graf nesnesi oluşturulmuştur. Bu işlem sırasında olası yinelenen kenarlar ve öz-döngüler (self-loops) otomatik olarak elenmiştir.

```
import networkx as nx
import pandas as pd

df = pd.read_csv("data/musae_git_edges.csv")
G = nx.from_pandas_edgelist(df, source="id_1", target="id_2")
```

2.4. Temel İstatistikler

Aşağıdaki **Tablo 2.1'**de ağın temel yapısal özellikleri özetlenmiştir. Değerler, veri setinin NetworkX'e aktarılmasının ardından hesaplanmıştır.

Kullanılan ağ tek bir bağlı bileşenden oluşmaktadır. Bu değer herhangi iki düğüm arasında mutlaka bir yol olduğunu göstermektedir. Ağın düşük yoğunluklu olması ise ağın seyrek (sparse) bir yapıda olduğunu göstermektedir. Bu seyrek yapı ağın gerçek dünya sosyal ağlarına benzediğini gösterir.

Ağın geçişkenlik değerinin düşük olması ise ağda ortak arkadaş ilişkilerinin zayıf olduğunu göstermektedir. Ağdaki düğümlerin yaklaşık %25-30'u Machine Learning Developer (Makine Öğrenmesi Geliştiricisi), %70-75'i Web Developer (WEB Geliştiricisi) olarak etiketlenmiştir. Bu heterojen yapıyı, ilerleyen bölümlerde yapacağımız "**8. TOPLULUK ANALİZİ**" kısmında yorumlayacağız.

Tablo 2.1. Kullanılan datasetin ve oluşturulan ağın temel yapısı

Özellik	Değer
Düğüm Sayısı (N)	37700
Düğüm Özellikleri	Var (bu çalışmada kullanılmamıştır).
Kenar Sayısı (L)	289003
Kenar Özellikleri	Yok
Ortalama Derece <k>	15.33
Yoğunluk	0.0004
Geçişkenlik (Transitivity)	0.0124
Bağlı Bileşen Sayısı	1

```
print(f"node sayisi (N): {G.number_of_nodes()}")
print(f"edge sayisi (L): {G.number_of_edges()}")
print(f"ortalama derece <k>: {sum(dict(G.degree()).values()) /
G.number_of_nodes():.2f}")
print(f"yogunluk: {nx.density(G):.4f}")
print(f"geciskenlik: {nx.transitivity(G):.4f}")
print(f"bagli bilesen sayisi: {nx.number_connected_components(G)}")
```

3. YÖNTEM VE KULLANILAN ARAÇLAR

Bu çalışmada yapılan analizler Python (versiyon: 3.11.15) programlama dili kullanılarak gerçekleştirilmiştir. Bu analizler için kullanılan başlıca kütüphaneler ve amaçları aşağıdaki **Tablo 3.1**'de verilmiştir.

Geliştirme ortamı olarak Visual Studio Code içinde Jupyter Notebook kullanılmış, analizler Windows Subsystem for Linux (WSL; Windows için Linux Alt Sistemi) üzerinde çalışan bir Conda sanal ortamında yürütülmüştür.

Analiz yaklaşımı olarak her bölümde gerçek ağ üzerinde hesaplanan değerler, aynı düğüm sayısı (N = 37.700) ve ortalama derece (⟨k⟩ = 15,33) kullanılarak üretilen Erdős–Rényi rastgele ağı **[5]** ile karşılaştırılmıştır. Böylelikle 37700 kişi birbirini tamamen rastgele takip etseydi ağın nasıl görüneceği incelenmiştir.

Tablo 3.1. Analizde kullanılan kütüphaneler

Kütüphane	Versiyon	Kullanım Amacı
NetworkX	3.6.1	Graf oluşturma, yapısal analiz
NumPy	2.4.4	Sayısal Hesaplamalar
SciPy	1.17.1	İstatistiksel hesaplamalar, eğri uydurma
Pandas	3.0.2	Veri okuma ve işleme
Matplotlib	3.10.8	Grafik oluşturma
Community	0.16	Topluluk tespiti (Louvain algoritması)
collections (Counter, defaultdict)	Standart kütüphane	Derece sayımı ve gruplama
scipy.stats	SciPy içinde	Güç yasası regresyonu

4. DERECE DAĞILIMI ANALİZİ

4.1. Yöntem

Derece dağılımı, ağdaki her düğümün komşu sayısı (derecesi) hesaplanarak elde edilmiştir. Olasılık dağılımı P(k) şu şekilde tanımlanır: k derecesine sahip düğüm sayısının toplam düğüm sayısına oranı.

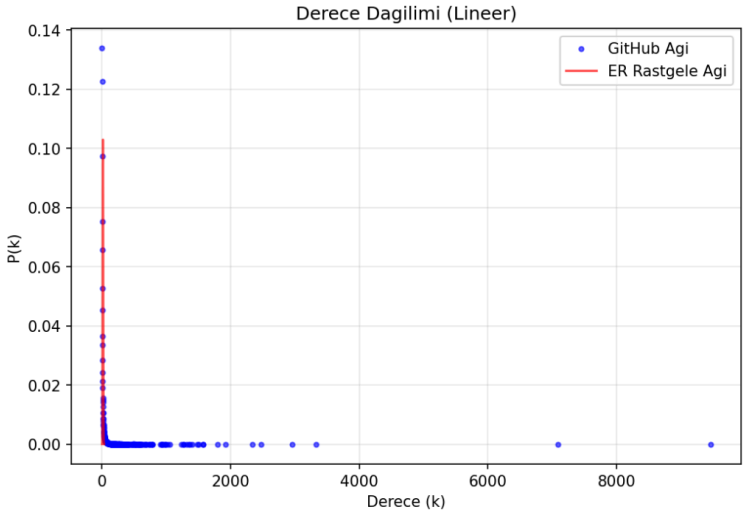
Karşılaştırma amacıyla aynı N ve ⟨k⟩ değerlerine sahip bir Erdős–Rényi rastgele ağı üretilmiştir.

Güç yasası $P(k) \sim k^{-\gamma}$ ifadesi logaritmik dönüşüm uygulandığında $log\, P(k) = -\gamma \cdot log\, k + C$ biçiminde doğrusal hale gelir. Bu nedenle güç yasası üsteli γ, log-log uzayında doğrusal regresyon kullanılarak tahmin edilmiştir.

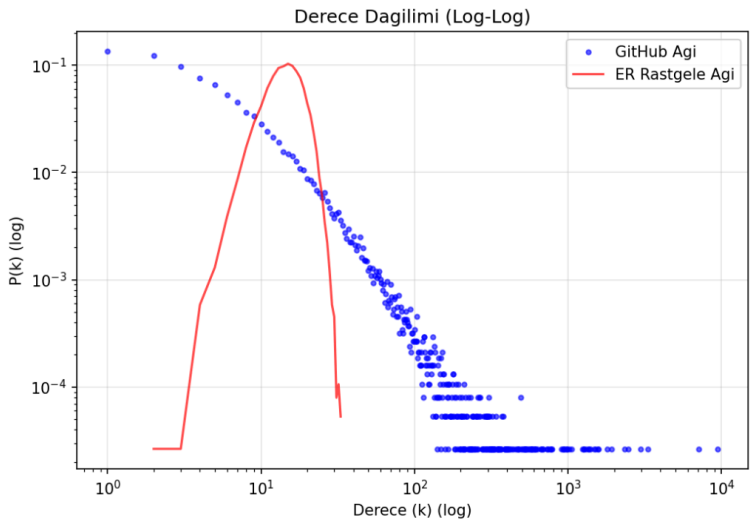
```
degrees = [d for _, d in G.degree()]
degree_count = collections.Counter(degrees)
deg_values = sorted(degree_count.keys())
deg_probs = [degree_count[k] / G.number_of_nodes() for k in
deg_values]

n = G.number_of_nodes()
p = (2 * G.number_of_edges()) / (n * (n - 1))  # baglanma olasiligi
ER = nx.erdos_renyi_graph(n, p, seed=42)
er_degrees = [d for _, d in ER.degree()]
er_count = collections.Counter(er_degrees)
er_values = sorted(er_count.keys())
er_probs = [er_count[k] / n for k in er_values]

from scipy import stats

# sadece k >= 1 olan degerleri al
deg_vals = np.array(deg_values)
deg_p = np.array(deg_probs)

mask = (deg_vals > 0) & (deg_p > 0)
log_k = np.log(deg_vals[mask])
log_p = np.log(deg_p[mask])

# dogrusal fit (log-log'da duz cizgi = power law)
slope, intercept, r_value, p_value, std_err = stats.linregress(log_k,
log_p)
```

4.2. Sonuçlar

Şekil 4.1'de lineer ve log-log ölçekli derece dağılımı grafikleri, **Şekil 4.2**'de ise power-law fit sonucu verilmektedir. Gerçek ağ ile ER rastgele ağının karşılaştırmasını ise **Tablo 4.1**'de bulabilirsiniz.

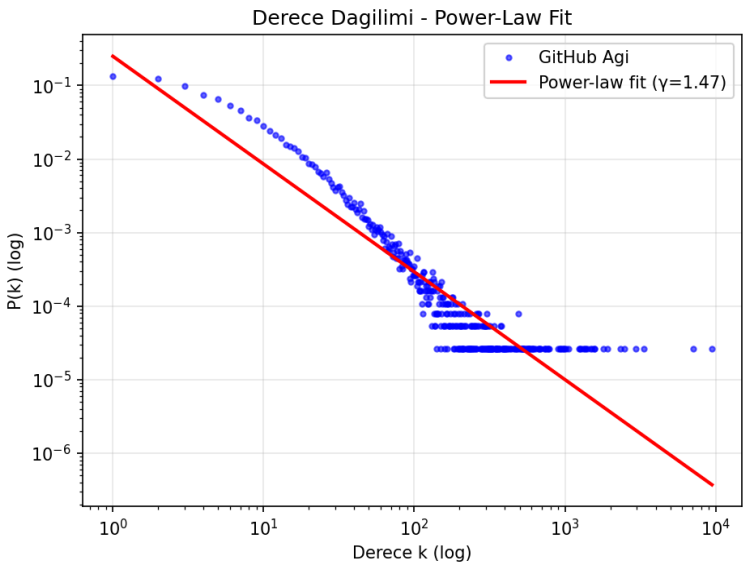


a)



b)

Şekil 4.1. Derece dağılım grafikleri: lineer ölçekli (**a**), log-log ölçekli (**b**)



Şekil 4.2. Derece dağılım grafiğinde power law fit

Tablo 4.1. Gerçek ağ ile ER rastgele ağının karşılaştırılması

Ölçüt	Gerçek Ağ	ER Rastgele Ağı
Maksimum Derece	9458	33
Minimum Derece	1	2
γ Tahmini	1.47	-
R^2	0.83	-

4.3. Karşılaştırma

ER rastgele ağında derece dağılımı, ⟨k⟩ = 15,33 civarında tepelenen Poisson dağılımına uymakta ve maksimum derece 33 ile ortalama derecenin yakınında kalmaktadır. Buna karşın GitHub ağında düğüm dereceleri 1 ile 9458 arasında geniş bir aralığa yayılmaktadır. Bu durum, GitHub ağının büyük çoğunluğunun az bağlantılı olduğunu ve az sayıda olan hub düğümlerinin binlerce bağlantıya sahip olduğunu göstermektedir.

4.4. Yorum

Şekil 4.1.a’daki lineer grafikte ağ sol tarafa yığılmış ve kuyruğu uzundur. Yani çoğu kişinin az bağlantısı ve az sayıda kişinin ise çok bağlantısı vardır.

Şekil 4.1.b’deki log-log grafikte gözlemlenen düz çizgiye yakın iniş, GitHub sosyal ağının scale-free özellikler taşıdığına işaret etmektedir. Tahmin edilen $\gamma \approx 1{,}47$ değeri, teorik olarak beklenen $2 < \gamma < 3$ aralığının altında kalmaktadır. Bu sapmanın temel nedeni, $k > 100$ bölgesinde düğümlerin beklenen power-law eğrisinden koparak yatay bir plato oluşturmasıdır. $\gamma < 2$ ($\gamma \approx 1{,}47$) olduğunda derece dağılımının ikinci momenti ıraksar. Bu durum ağda çok yüksek dereceli düğümlerin varlığını açıklar ve gözlemlenen $k_{max} = 9458$ değeriyle tutarlıdır. Bununla birlikte maksimum derece farkı (GitHub: 9458; ER: 33) ve dağılımın uzun kuyruklu yapısı, ağın scale-free özellikli olduğunu desteklemektedir.

Şekil 4.2'de log-log ölçekte uygulanan doğrusal regresyon fit'i $\gamma \approx 1{,}47$ ($R^2 = 0{,}83$) sonucunu vermiştir. Fit eğrisi düşük ve orta dereceli bölgede ($k < 100$) veriye iyi uymakta, ancak yüksek dereceli bölgede ($k > 100$) gözlemlenen plato nedeniyle sapmaktadır. $R^2 = 0{,}83$ değeri fit'in kısmi uyumunu göstermekte olup tam bir power-law yerine yaklaşık bir güç yasası davranışına işaret etmektedir.

Bu yapı, ağın oluşumunda tercihli bağlanma (preferential attachment) mekanizmasının etkili olabileceğini düşündürmektedir. Yani yeni katılan düğümlerin zaten yüksek bağlantıya sahip düğümlere bağlanma eğilimi göstermesi gözlemlenen hub yapısını açıklamaktadır.

Derece dağılımının ortaya koyduğu bu hub ağırlıklı yapı, ağdaki mesafe dinamiklerini doğrudan etkiler. Bir sonraki bölümde bu etkinin ortalama yol uzunluğu üzerindeki yansımaları incelenmektedir.

5. MESAFE VE KÜÇÜK DÜNYA ANALİZİ

5.1. Yöntem

NetworkX tek iş parçacıklı (single-threaded) çalıştığından ve 37700 düğümlü ağda tüm çiftler arasındaki mesafeyi hesaplamak (N×(N-1)/2≈710 milyon çift) saatler alacağından, ortalama yol uzunluğu rastgele seçilen 5000 kaynak düğüm üzerinden örnekleme yöntemiyle tahmin edilmiştir. N = 37700 düğümlü ağda 5000 düğüm örneklenmesi, toplam düğümlerin yaklaşık %13.3'üne karşılık gelmektedir. Farklı örnekleme boyutlarıyla yapılan ön denemeler sonucunda 5000 değeri yeterli ve kararlılık sağlamak amacıyla uygun olarak seçilmiştir.

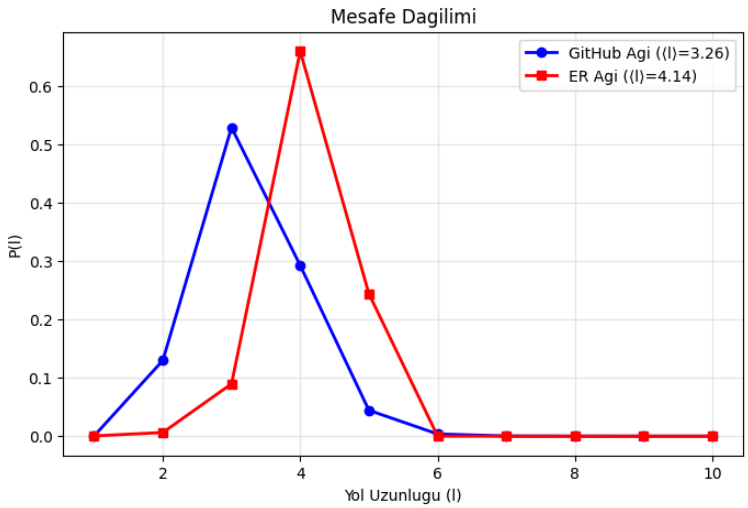
GitHub ağındaki en kısa yol uzunlukları, NetworkX kütüphanesinin "`single_source_shortest_path_length`" fonksiyonu kullanılarak hesaplanmıştır.

```
import random
import time

random.seed(42)
sample_nodes = random.sample(list(G.nodes()), 5000)

print(f"ornekleme basladi: {len(sample_nodes)} node")
start = time.time()

lengths = []
for i, node in enumerate(sample_nodes):
    path_lengths = nx.single_source_shortest_path_length(G, node)
    lengths.extend(path_lengths.values())
    if (i+1) % 500 == 0:
        elapsed = time.time() - start
        print(f"  {i+1}/5000 tamamlandi — {elapsed:.0f} saniye
gecti")

# 0 (kendisine olan mesafe) cikar
lengths = [l for l in lengths if l > 0]
```

```
avg_path = np.mean(lengths)
max_path = max(lengths)

# ER agi icin de ayni hesaplamalar
er_sample = random.sample(list(ER.nodes()), 5000)
er_lengths = []
for node in er_sample:
    pl = nx.single_source_shortest_path_length(ER, node)
    er_lengths.extend(pl.values())
er_lengths = [l for l in er_lengths if l > 0]

# mesafe dagilimi grafigi
from collections import Counter

len_count = Counter(lengths)
er_len_count = Counter(er_lengths)

all_dists = sorted(set(lengths) | set(er_lengths))

git_probs = [len_count.get(d, 0) / len(lengths) for d in all_dists]
er_probs_dist = [er_len_count.get(d, 0) / len(er_lengths) for d in
all_dists]
```

5.2. Sonuçlar

Şekil 5.1'de GitHub ağı ile ER rastgele ağının mesafe dağılımı karşılaştırmalı olarak verilmektedir. **Tablo 5.1**'de ise iki ağın temel mesafe ölçütleri özetlenmektedir.



Şekil 5.1. Mesafe dağılımı grafiği

Tablo 5.1. Gerçek ağ ile ER rastgele ağının karşılaştırılması

Ölçüt	Gerçek Ağ	ER Rastgele Ağı
Ortalama yol uzunluğu	3.26	4.14
Maksimum mesafe	10	6

5.3. Karşılaştırma

ER rastgele ağında mesafe dağılımı l = 4 değerinde keskin bir tepe oluştururken, GitHub ağında dağılım l = 3 civarında yoğunlaşmakta ve daha geniş bir aralığa yayılmaktadır. GitHub ağının ortalama yol uzunluğu (3,26), aynı N ve ⟨k⟩ değerlerine sahip ER ağından (4,14) belirgin biçimde daha kısadır.

Öte yandan maksimum mesafe GitHub ağında daha yüksek çıkmıştır (GitHub: 10; ER: 6). Bu durum scale-free yapının bir sonucudur. Hub düğümleri sayesinde çoğu çift kısa yollarla birbirlerine ulaşırken, düşük dereceli uç düğümler daha fazla adım atmak zorunda kalmaktadır.

5.4. Yorum

GitHub sosyal ağı, küçük dünya özelliği sergilemektedir. **Şekil 5.1**'de görüldüğü üzere mesafe dağılımı l = 3 civarında yoğunlaşmakta olup 37.700 düğümlü bu ağda herhangi iki geliştirici arasındaki ortalama mesafe yalnızca 3,26 adımdır. Bu sonuç, açık kaynak yazılım geliştirme ekosisteminde bilginin ve işbirliğinin ne kadar hızlı yayılabileceğini göstermektedir.

Tablo 5.1'de görülen maksimum mesafe değeri (10), örnekleme yöntemi kullanıldığından gerçek diameter için bir alt sınır tahmini olarak değerlendirilmiştir. Diameter, ağın en uzak iki düğümü arasındaki en kısa yol olarak tanımlanır ve bu değerin tam hesaplanması ancak tüm düğüm çiftleri nin incelenmesi ile yapılabilir.

Ayrıca, ağın düşük yoğunluğa rağmen kısa ortalama yol uzunluğuna sahip olması ve kümeleşme eğilimi göstermesi, küçük dünya ağlarının karakteristik özellikleri ile uyumludur.

Ağın hub düğümlerle açıklanan kısa mesafeli yapısına ek olarak bu düğümlerin komşuluk örüntülerini anlamak da önemlidir. Bir sonraki bölümde derece korelasyonları incelenerek hub düğümlerin kiminle bağlantı kurduğu sorusu yanıtlanmaktadır.

6. DERECE KORELASYONLARI

6.1. Yöntem

Derece korelasyonları üç ölçüt ile incelenmiştir:

- Kenar uç noktalarının derece çiftlerinin ortak olasılık dağılımı olan e_ij matrisi, her kenar için uç düğümlerin derece çiftleri sayılarak ve toplam kenar sayısına bölünerek hesaplanmıştır.
- Ortalama en yakın komşu derecesi k_nn(k), NetworkX'in “`average_neighbor_degree`“ fonksiyonu ile hesaplanmıştır.

- Assortativity katsayısı r ise hem Newman formülasyonu **[4]** kullanılarak e_ij matrisinden manuel olarak türetilmiş, hem de NetworkX'in `degree_assortativity_coefficient` fonksiyonu ile doğrulanmıştır.

Xulvi-Brunet & Sokolov (XBS) algoritması **[6]** ile aynı derece dizisini koruyan maksimum assortative ve maksimum disassortative ağlar üretilmiştir ve GitHub ağının bu iki uç durum arasındaki konumu incelenmiştir.

```
# assortativity katsayisi
r = nx.degree_assortativity_coefficient(G)
print(f"Assortativity katsayisi (r): {r:.4f}")

# k_nn(k): ortalama komsu derecesi
knn = nx.average_neighbor_degree(G)

# k degerlerine gore gruplama yap
from collections import defaultdict
knn_by_degree = defaultdict(list)
for node, avg_neighbor_deg in knn.items():
    k = G.degree(node)
    knn_by_degree[k].append(avg_neighbor_deg)

# her k icin ortalama al
k_vals = sorted(knn_by_degree.keys())
knn_vals = [np.mean(knn_by_degree[k]) for k in k_vals]

# eij matrisi hesaplama
degrees = dict(G.degree())
max_degree = max(degrees.values())

# eij: kenar uclarinin derece ciftlerinin olasilik matrisi
from collections import defaultdict
edge_pairs = defaultdict(int)
total_edges = G.number_of_edges()

for u, v in G.edges():
    ki = degrees[u]
```

```
    kj = degrees[v]
    edge_pairs[(ki, kj)] += 1
    edge_pairs[(kj, ki)] += 1  # yonsuz ag

# normalize etme
eij = {pair: count / (2 * total_edges) for pair, count in
edge_pairs.items()}

# en sik gorulen 10 derece cifti
top_pairs = sorted(eij.items(), key=lambda x: x[1],
reverse=True)[:10]
print("En sık görülen derece çiftleri (ki, kj) → P(eij):")
for (ki, kj), prob in top_pairs:
    print(f"  ({ki:4d}, {kj:4d}) → {prob:.6f}")

# eij'den r hesapla (manuel, NetworkX ile teyit ederiz)
# r = [Σ_ij (ki*kj*eij) - μ²] / σ²
numerator = sum(ki * kj * p for (ki, kj), p in eij.items())
q_k = {}  # kenar ucu derece dagilimi
for (ki, kj), p in eij.items():
    q_k[ki] = q_k.get(ki, 0) + p
mu = sum(k * q for k, q in q_k.items())
sigma2 = sum(k**2 * q for k, q in q_k.items()) - mu**2

r_manual = (numerator - mu**2) / sigma2

# Xulvi-Brunet & Sokolov Algoritmasi
def xulvi_brunet_sokolov(G, assortative=True, steps=50000):
    """
    assortative=True  → yuksek dereceli node'lari birbirine bagla
    assortative=False → yuksek ile dusuk node'lari dugummleri bagla
    """
    H = G.copy()
    edges = list(H.edges())

    for _ in range(steps):
        # rastgele iki kenar secme
        if len(edges) < 2:
            break
        idx1, idx2 = random.sample(range(len(edges)), 2)
        u, v = edges[idx1]
        s, t = edges[idx2]
```

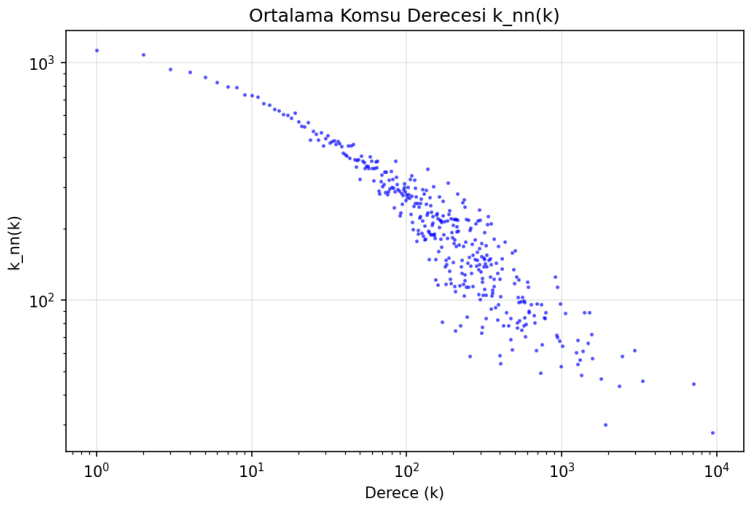
```
        # 4 node'un hepsi farkli olmali
        if len({u, v, s, t}) < 4:
            continue

        degrees = [H.degree(u), H.degree(v), H.degree(s),
H.degree(t)]
        nodes = [u, v, s, t]
        sorted_nodes = [x for _, x in sorted(zip(degrees, nodes),
reverse=True)]

        if assortative:
            # en yuksek iki node birbirine, en dusuk iki node
birbirine
            new_e1 = (sorted_nodes[0], sorted_nodes[1])
            new_e2 = (sorted_nodes[2], sorted_nodes[3])
        else:
            # en yuksek ile en dusuk, ortadakiler
            new_e1 = (sorted_nodes[0], sorted_nodes[3])
            new_e2 = (sorted_nodes[1], sorted_nodes[2])

        # kontrol
        if (new_e1[0] == new_e1[1] or new_e2[0] == new_e2[1]):
            continue
        if H.has_edge(*new_e1) or H.has_edge(*new_e2):
            continue

        # eski edgeleri sil, yenileri koy
        H.remove_edge(u, v)
        H.remove_edge(s, t)
        H.add_edge(*new_e1)
        H.add_edge(*new_e2)
        edges[idx1] = new_e1
        edges[idx2] = new_e2

    return H

# Assortative Ag
G_assort = xulvi_brunet_sokolov(G, assortative=True, steps=5000000)
r_assort = nx.degree_assortativity_coefficient(G_assort)

# Disassortative Ag
```

```
G_disassort = xulvi_brunet_sokolov(G, assortative=False,
steps=5000000)
r_disassort = nx.degree_assortativity_coefficient(G_disassort)
```

6.2. Sonuçlar

Şekil 6.1'de k_nn(k) grafiği verilmektedir. **Tablo 6.1**'de e_ij matrisinin en sık görülen derece çiftleri, **Tablo 6.2**'de ise assortativity ölçütlerinin özeti sunulmaktadır.

Tablo 6.1'de verilen e_ij matrisinin en sık görülen derece çiftleri incelendiğinde, en yüksek olasılıklı çiftlerin hepsinin maksimum dereceli hub düğümü (k = 9458) içerdiği görülmektedir. Yönsüz ağda (k_i, k_j) ve (k_j, k_i) çiftleri eşdeğerdir bu nedenle tablodaki tekrarlar çıkarılmıştır.



Şekil 6.1. Ortalama en yakın komşu derecesi k_nn(k) grafiği (log-log ölçek)

Tablo 6.1. e_ij matrisinin en sık görülen derece çiftleri

Derece Çifti (k_i, k_j)	P(e_ij)
(9458, 2)	0.001024
(9458, 3)	0.000974
(9458, 4)	0.000957
(9458, 5)	0.000919
(9458, 6)	0.000822

Tablo 6.2. Assortativity ölçütleri özeti

Ölçüt	Değer
Assortativity katsayısı (r)	-0.0752
Manuel hesaplanan r (e_ij'den)	-0.0752
Assortative ağ - XBS (r)	-0.0593
Disassortative ağ - XBS (r)	-0.0883

6.3. Yorum

Tablo 6.2'de görüldüğü üzere r = -0,0752 değeri ağın hafif disassortative olduğuna işaret etmektedir. Yani yüksek dereceli hub düğümler ağırlıklı olarak düşük dereceli düğümlerle bağlantı kurmaktadır. Bu durum **Tablo 6.1**'deki e_ij tablosunda da açıkça görülmektedir. En olası kenar çiftlerinin tamamında bir uç maksimum dereceli hub (k = 9458) iken, diğer uç ise düşük dereceli bir düğümdür. **Şekil 6.1**'deki k_nn(k) grafiğinde de aynı örüntü gözlemlenmektedir. Derece arttıkça ortalama komşu derecesi azalmakta, yani yüksek dereceli düğümlerin komşuları giderek daha düşük dereceli olmaktadır.

Bu durum, yüksek dereceli kullanıcıların genellikle düşük dereceli kullanıcılarla etkileşime girdiğini göstermektedir. Sosyal ağlar için bu yapı, daha popüler ve merkezi konumdaki kullanıcıların geniş bir kitleye ulaşabildiği hiyerarşik bir organizasyonun varlığına işaret etmektedir.

6.4. Ek Analiz

Tablo 6.2'de verilen Xulvi-Brunet & Sokolov sonuçları incelendiğinde 5.000.000 adım uygulanmış olmasına rağmen üretilen assortative (r = -0,0593) ve disassortative (r = -0,0883) ağlar arasındaki fark oldukça dardır. Bu durum ağın seyrek yapısından (yoğunluk = 0,0004) kaynaklanmaktadır. Düşük yoğunlukta geçerli kenar değiş tokuşu bulmak güçleşmektedir. Algoritmanın ağı yeniden düzenleyebileceği alan kısıtlı kalmaktadır.

Bununla birlikte GitHub ağının (r = -0,0752) assortative ve disassortative uç noktalar arasında nötre yakın bir konumda yer aldığı görülmektedir. Teorik olarak XBS algoritması yeterli sayıda adımla çalıştırıldığında assortative ağın r değerinin pozitif değere yaklaşması beklenir. Ancak bu çalışmada bunun gözlenmemesi algoritmanın yetersizliğinden ziyade ağın seyrek yapısının, derece dizisi korunarak yapılabilecek yeniden bağlantıları ciddi biçimde sınırlamasının doğal bir sonucu olarak değerlendirilebilir.

7. GÜRBÜZLÜK ANALİZİ

7.1. Yöntem

Bu kısımda iki farklı düğüm silme stratejisi uygulanmıştır:

- Rastgele saldırıda her adımda rastgele düğümler silinmiştir.
- Hedefli saldırıda ise her adımda o anki en yüksek dereceli düğümler seçilerek silinmiştir.

Hedefli saldırıda kullanılan yaklaşım dinamiktir. Yani bu saldırıda her silme adımından sonra derece sıralaması yeniden hesaplanır. Tüm düğümlerin başta

bir kez sıralanmasıyla yapılan saldırıya kıyasla daha gerçekçi bir saldırı senaryosu oluşturulmuş ve ağın gerçek kırılganlığı daha doğru yansıtılmıştır.

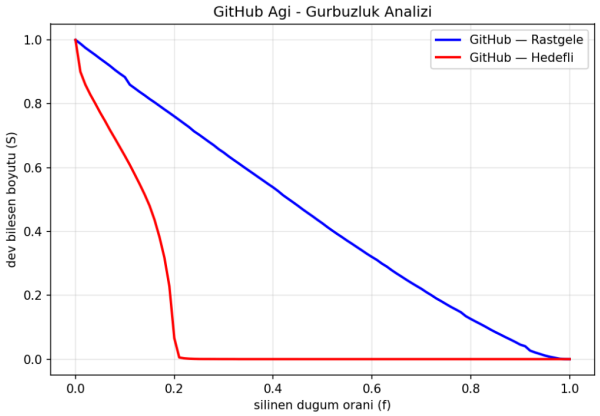
Her adımda ağın en büyük bağlı bileşeninin boyutu toplam düğüm sayısına oranlanarak kaydedilmiştir. Karşılaştırma için aynı analiz ER ağına da uygulanmıştır. Kritik eşik değeri (f_c), dev bileşen boyutunun %1'in altına düştüğü silme oranı olarak tanımlanmıştır.

```
def robustness_analysis(G, attack_type='random', steps=100):
    """
    attack_type = 'random'  → rastgele düğüm sil
    attack_type = 'targeted' → en yüksek dereceli düğümü sil
    """
    H = G.copy()
    n = H.number_of_nodes()

    giant_sizes = [1.0]  # baslangic
    fractions = [0.0]

    step_size = n // steps

    for i in range(steps):
        if H.number_of_nodes() == 0:
            break

        # silinecek dugumleri secme
        if attack_type == 'random':
            nodes_to_remove = random.sample(list(H.nodes()),
min(step_size, H.number_of_nodes()))
        else:  # targeted
            sorted_nodes = sorted(H.degree(), key=lambda x: x[1],
reverse=True)
            nodes_to_remove = [n for n, _ in
sorted_nodes[:step_size]]

        H.remove_nodes_from(nodes_to_remove)

        if H.number_of_nodes() == 0:
```

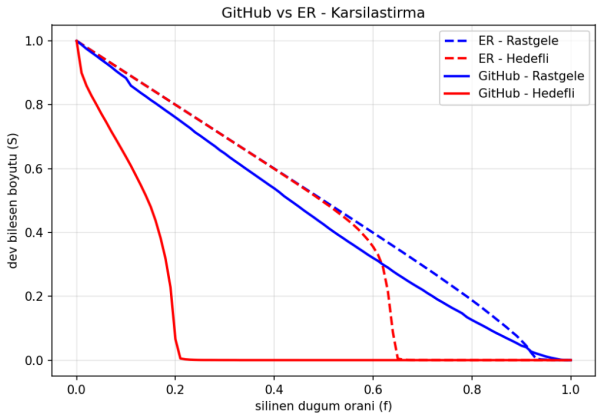
```
            giant_sizes.append(0)
        else:
            largest_cc = max(nx.connected_components(H), key=len)
            giant_sizes.append(len(largest_cc) / n)

        fractions.append((i + 1) * step_size / n)
    return fractions, giant_sizes

f_random, g_random = robustness_analysis(G, attack_type='random',
steps=100)

f_targeted, g_targeted = robustness_analysis(G,
attack_type='targeted', steps=100)

f_er_random, g_er_random = robustness_analysis(ER,
attack_type='random', steps=100)

f_er_targeted, g_er_targeted = robustness_analysis(ER,
attack_type='targeted', steps=100)

# kritik esik: dev bilesenin %1'e dustugu nokta
def find_critical_threshold(fractions, giant_sizes, threshold=0.01):
    for f, s in zip(fractions, giant_sizes):
        if s <= threshold:
            return f
    return 1.0

fc_random = find_critical_threshold(f_random, g_random)
fc_targeted = find_critical_threshold(f_targeted, g_targeted)
```

7.2. Sonuçlar

Şekil 7.1.a'da GitHub ağının kendi içinde rastgele ve hedefli saldırı karşılaştırması, **Şekil 7.1.b**'de ise GitHub ağı ile ER rastgele ağının dört saldırı senaryosu birlikte verilmiştir. **Tablo 7.1**'de her iki ağ için hesaplanan kritik eşik değerleri özetlenmektedir.



a)



b)

Şekil 7.1. Gürbüzlük analizi grafikleri: GitHub ağı (**a**), GitHub vs ER karşılaştırması (**b**)

Tablo 7.1. GitHub ve ER ağları için kritik eşik değerleri (f_c)

Saldırı Türü	GitHub f_c	ER f_c
Rastgele	0.97	0.93
Hedefli	0.21	0.62

7.3. Kritik Eşik Analizi

Tablo 7.1'de görüldüğü üzere GitHub ağı rastgele düğüm silmeye karşı son derece dayanıklıdır. Ağın en büyük bağlı bileşeni ancak düğümlerin %97'si silindiğinde çökmektedir. Buna karşın hedefli saldırıda yalnızca %21'lik bir silme oranı ağı işlevsiz kılmaktadır. ER ağında ise iki saldırı türü arasındaki fark (Δf_c = 0.93 - 0.62 = 0.31), GitHub ağındaki farka (Δf_c = 0.97 - 0.21 = 0.76) kıyasla çok daha küçüktür. Bu sayısal fark, hub yapısının ağa kazandırdığı kırılganlığı ortaya koyar.

7.4. Yorum

Tablo 7.1 ve **Şekil 7.1**'deki bulgular birlikte değerlendirildiğinde, scale-free ağların bilinen "sağlamlık-kırılganlık paradoksunu" açıkça ortaya çıkmaktadır. Hub düğümlerin varlığı ağı rastgele saldırılara karşı güçlü kılarken, aynı zamanda hedefli saldırılara karşı zayıf noktası haline gelmektedir. ER ağında hub bulunmadığından her iki saldırı türüne karşı davranış benzerdir.

GitHub ekosisteminde çok bağlantılı geliştiricilerin platformdan ayrılması işbirliği ağını ciddi biçimde etkileyebilirken rastgele kullanıcı kayıpları ağın genel bağlantı yapısını önemli ölçüde bozmaz.

Elde edilen sonuçlar, ağın rastgele düğüm kayıplarına karşı dayanıklı ancak yüksek dereceli düğümlere yönelik hedefli saldırılara karşı oldukça hassas olduğunu göstermektedir. Bu davranış, scale-free ağların tipik bir özelliği olup ağın yapısal olarak hub düğümlere bağımlı olduğunu ortaya koymaktadır.

Gürbüzlük analizi ağın hub düğümlerine olan yapısal bağımlılığını ortaya koymuştur. Bu bağımlılığın sosyal boyutunu anlamak için bir sonraki bölümde topluluk yapısı incelenmektedir.

8. TOPLULUK ANALİZİ

8.1. Yöntem

Topluluk tespiti için Louvain algoritması **[3]** kullanılmıştır. Bu algoritma, modularity değerini artıracak şekilde ağı birbirine daha sıkı bağlı gruplara ayırmaktadır. Girvan-Newman algoritması ise 37.700 düğümlü bu ağda kullanılması için hesaplama maliyeti çok yüksek olacağından tercih edilmemiştir.

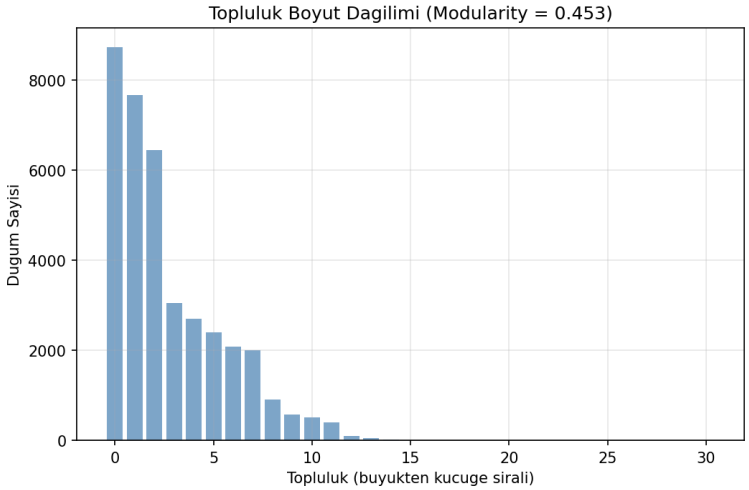
Ek analiz olarak toplulukların demografik kompozisyonu `target.csv` etiketleri kullanılarak incelenmiştir.

```
import community as community_louvain

# Louvain community detection
partition = community_louvain.best_partition(G, random_state=42)

# Modularity degeri
modularity = community_louvain.modularity(partition, G)

# Kac topluluk var
num_communities = len(set(partition.values()))

# Topluluk boyutlari
from collections import Counter
community_sizes = Counter(partition.values())
sizes = sorted(community_sizes.values(), reverse=True)

# target.csv ile topluluk karsilastirmasi
df_target = pd.read_csv("data/musae_git_target.csv")

# Her topluluktaki web/ML developer orani
df_target['community'] = df_target['id'].map(partition)

community_composition =
df_target.groupby('community')['ml_target'].agg(['sum', 'count'])
community_composition['ml_ratio'] = community_composition['sum'] /
community_composition['count']
```

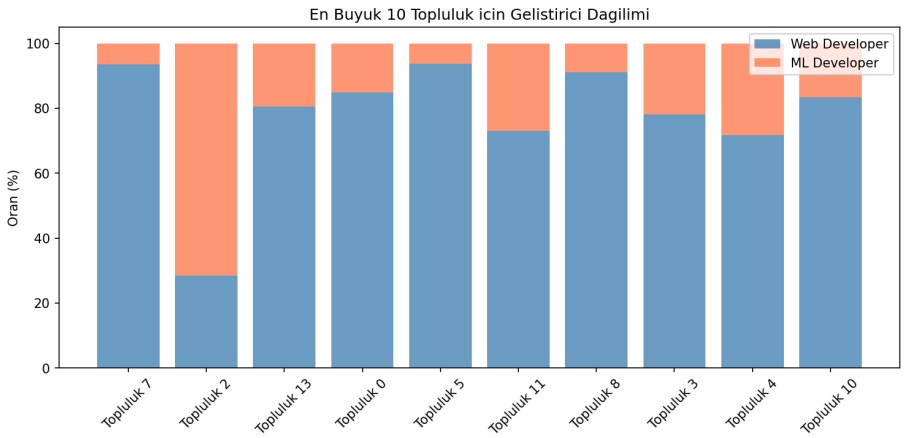
```
community_composition['web_ratio'] = 1 -
community_composition['ml_ratio']
community_composition = community_composition.sort_values('count',
ascending=False)
```

8.2. Sonuçlar

Şekil 8.1'de Louvain algoritmasının tespit ettiği toplulukların boyut dağılımı, **Şekil 8.2**'de ise en büyük 10 topluluğun Web/ML geliştirici dağılımı verilmektedir. **Tablo 8.1**'de temel topluluk istatistikleri özetlenmektedir.



Şekil 8.1. Topluluk boyut dağılımı - Louvain algoritması (Modularity = 0,453)



Şekil 8.2. En büyük 10 topluluğun Web/ML geliştirici dağılımı

Tablo 8.1. Louvain algoritması topluluk analizi sonuçları

Ölçüt	Değer
Topluluk sayısı	31
Modularity	0.453
En büyük topluluk	8732 düğüm
Ortalama topluluk boyutu	1216

8.3. Küme Analizi

Tablo 8.1'de görüldüğü üzere modularity değeri 0.45'tir. Bu güçlü bir topluluk yapısına işaret etmektedir. 0.3'ün üzerindeki değerler anlamlı bir topluluk yapısı olarak kabul edilmektedir. **Şekil 8.1**'deki boyut dağılımı incelendiğinde 31 topluluktan ilk 10'u ağın büyük çoğunluğunu (%97'sini) oluşturmaktadır. En büyük topluluk 8.732 düğüm ile ağın yaklaşık %23'ünü kapsamaktadır. **Şekil 8.1**'de ayrıca boyut dağılımının sağa dayalı yapısı dikkat çekmektedir. Birkaç büyük topluluk ağın büyük bölümünü oluştururken geri kalan topluluklar çok daha küçük kalmaktadır.

8.4. Ek Analiz

Şekil 8.2'de verilen toplulukların "`target.csv`" etiketleriyle karşılaştırılmasını gösteren sütun grafiği ilginç bir örüntü ortaya koymaktadır. **Tablo 8.2**'de görüldüğü üzere toplulukların büyük çoğunluğu web developer ağırlıklı iken "Topluluk 2", %71,5 ML developer oranıyla diğer tüm topluluklardan belirgin biçimde ayrışmaktadır. Bu bulgu, makine öğrenmesi geliştiricilerinin GitHub sosyal ağında kendi aralarında kümelenme eğilimi gösterdiğine işaret etmektedir.

Tablo 8.2. En büyük 5 topluluğun Web/ML geliştirici dağılımı

Topluluk	Boyut	ML Dev	WEB Dev
7	8732	% 6.5	% 93.5
2	7664	% 71.5	% 28.5
13	6454	% 19.4	% 80.6
0	3047	% 15.1	% 84.9
5	2702	% 6.2	% 93.8

8.5. Yorum

Tablo 8.1'deki modularity değeri (0,453) ve **Şekil 8.1**'deki boyut dağılımı birlikte değerlendirildiğinde, Louvain algoritmasının GitHub ağında güçlü bir topluluk yapısı tespit ettiği görülmektedir. **Tablo 8.2** ve **Şekil 8.2**'de öne çıkan en dikkat çekici bulgu "Topluluk 2"dir: %71,5 ML developer oranı, bu topluluğun yalnızca sosyal değil mesleki bir kümelenmeyi de yansıttığına işaret etmektedir. Bu geliştiricilerin benzer makine öğrenmesi repoları üzerinde çalışması ve birbirini takip etmesi sonucunda doğal bir küme oluşturduğu düşünülmektedir. Bu bulgu, ağ yapısının yalnızca sosyal ilişkileri değil, mesleki uzmanlaşmayı da yansıttığını göstermektedir.

9. GENEL DEĞERLENDİRME VE SONUÇ

9.1. Tüm Bulguların Özeti

Bu çalışmada 37.700 düğüm ve 289.003 kenardan oluşan GitHub sosyal ağı, ağ biliminin temel kavramları çerçevesinde analiz edilmiştir. Elde edilen bulgular aşağıdaki **Tablo 9.1**'de özetlenmektedir.

Tablo 9.1. Tüm analiz bölümlerinin bulgu özeti

Analiz	Temel Bulgu	Sonuç
Derece Dağılımı	γ ≈ 1,47, k_max = 9458	Scale-free özellik var
Mesafe & Küçük Dünya	⟨l⟩ = 3,26, diameter ≈ 10	Small-world özellik var
Derece Korelasyonları	r = -0,0752	Hafif disassortative
Gürbüzlük	f_c(rastgele) = 0,97, f_c(hedefli) = 0,21	Sağlamlık-kırılganlık paradoksu
Topluluk	31 topluluk, modularity = 0,453	Güçlü topluluk yapısı

9.2. Ağın Karakteristik Özellikleri

Tüm bulgular bir arada değerlendirildiğinde GitHub sosyal ağının üç temel karakteristik özellik sergilediği görülmektedir.

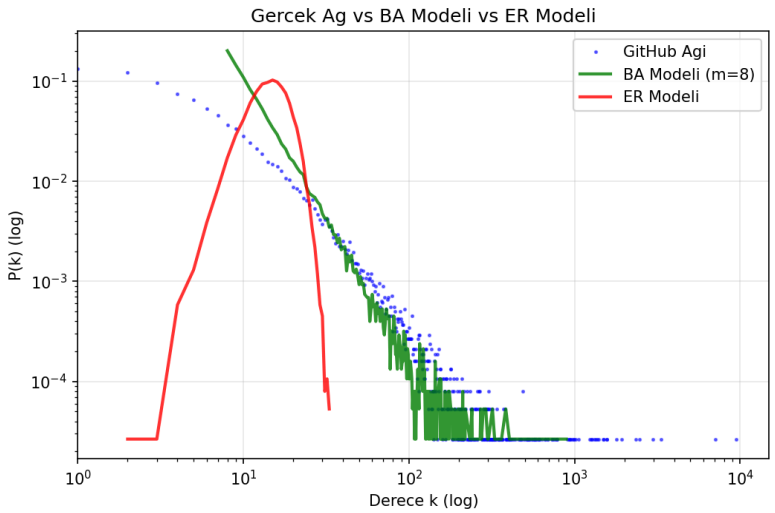
- **Scale-free yapı:** "**4. DERECE DAĞILIMI ANALİZİ**" bölümünde gösterildiği üzere derece dağılımı power-law eğilimi sergilemekte ve ER rastgele ağından belirgin biçimde ayrılmaktadır. k_max = 9458 değeri ER ağının maksimum derecesinin (33) yaklaşık 286 katıdır.

- **Küçük dünya özelliği:** "**5. MESAFE VE KÜÇÜK DÜNYA ANALİZİ**" bölümünde hesaplanan ortalama yol uzunluğu ⟨l⟩ = 3,26 olan bu değer, ER ağından (4,14) kısa olup hub düğümlerin ağdaki köprü rolünü ortaya koymaktadır.

- **Güçlü topluluk yapısı: "8. TOPLULUK ANALİZİ"** bölümünde modularity 0,453 ile tespit edilen 31 topluluk, ağın yalnızca sosyal değil mesleki uzmanlaşmayı da yansıttığını göstermektedir. Topluluk 2'nin %71,5 ML developer ağırlıklı yapısı bu durumun somut kanıtıdır.

9.3. Ağ Modeli Karşılaştırması

Şekil 9.1'de gerçek GitHub ağının derece dağılımı, Barabási-Albert (BA) modeli **[1]** ve Erdős-Rényi (ER) modeli ile karşılaştırmalı olarak verilmektedir.



Şekil 9.1. Gerçek ağ, BA modeli ve ER modeli derece dağılımı karşılaştırması (log-log ölçek)

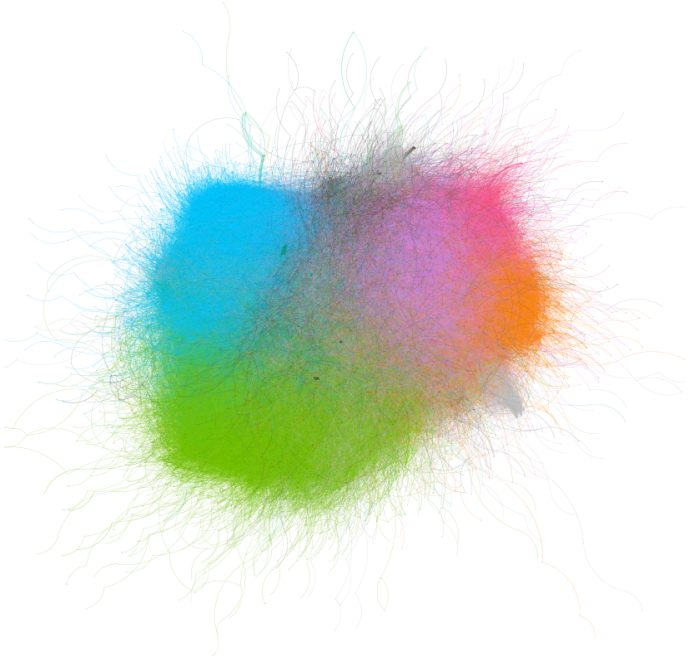
Şekil 9.1 incelendiğinde GitHub ağının BA modeliyle ER modeline kıyasla çok daha uyumlu olduğu görülmektedir. ER modelinin Poisson dağılımı gerçek ağın uzun kuyruklu yapısını hiç yakalayamazken, BA modeli özellikle orta derece bölgesinde ($10 < k < 100$) gerçek ağa yakın bir dağılım sergilemektedir. Bu durum GitHub geliştiricilerinin platforma katılırken tercihli bağlanma

(preferential attachment) mekanizmasıyla yani popüler geliştiricileri takip ederek ağa dahil olduğunu düşündürmektedir.

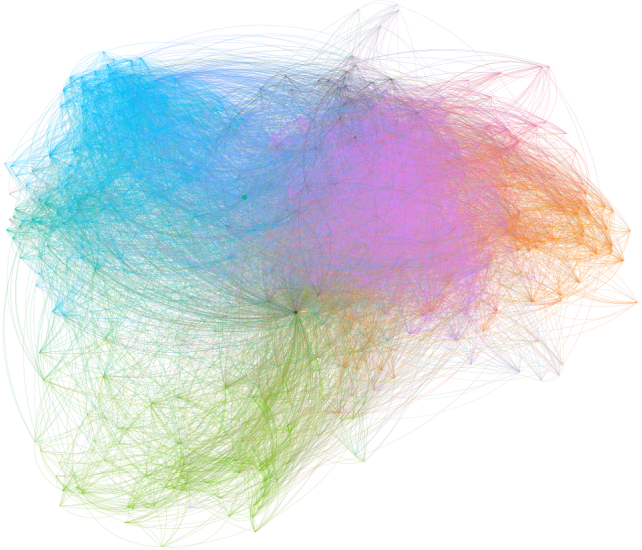
```
# BA modeli: her yeni dugum L tane kenar ekler
# Ortalama derece = 2L -> L = ⟨k⟩/2 = 15.33/2 ~ 8
L_BA = round(sum(dict(G.degree()).values()) / (2 *
G.number_of_nodes()))
print(f"BA modeli icin m = {L_BA}")

BA = nx.barabasi_albert_graph(n=G.number_of_nodes(), m=L_BA, seed=42)

# Karsilastirma
r_ba = nx.degree_assortativity_coefficient(BA)

# BA dagilimini duzlestirme
ba_degrees = [d for _, d in BA.degree()]
ba_count = Counter(ba_degrees)
ba_vals = sorted(ba_count.keys())
ba_probs_raw = np.array([ba_count[k] / BA.number_of_nodes() for k in
ba_vals])
```

9.4. Ağ Görselleştirmesi

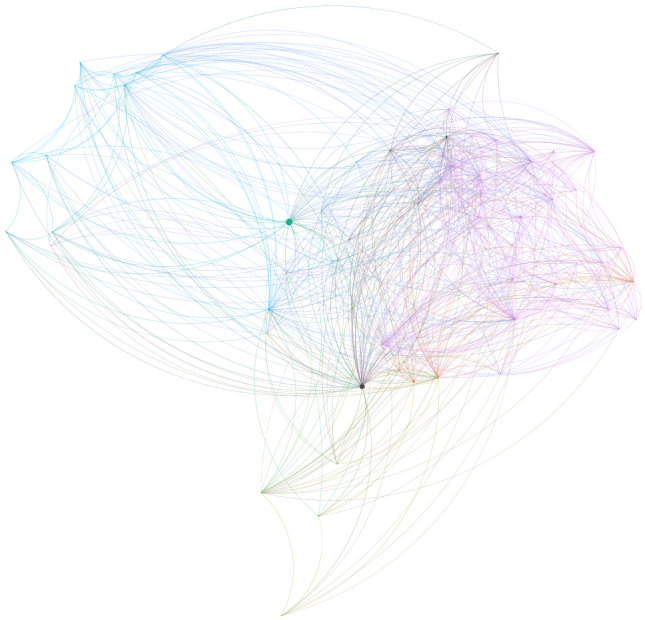
Şekil 9.2'de Gephi yazılımı **[8]** kullanılarak oluşturulan ağ görselleştirmeleri verilmektedir. ForceAtlas2 layout algoritması uygulanmış, düğüm boyutları dereceyle orantılı olarak ayarlanmış ve renkler Louvain algoritmasıyla tespit edilen topluluk sınıflarına göre atanmıştır. Görselleştirme amacıyla okunabilirliği artırmak için derece filtresi uygulanmıştır. Tüm analizler 37.700 düğümlü tam ağ üzerinde gerçekleştirilmiştir.



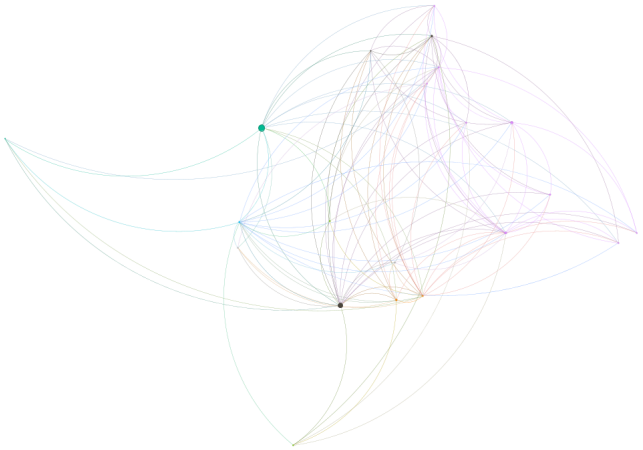
a)



b)



c)



d)

Şekil 9.2. GitHub ağı genel görünümü: tüm düğümler (Gephi, ForceAtlas2) (**a**), yüksek dereceli düğümler (derece > 100 filtresi uygulanmış) (**b**), yüksek dereceli düğümler (derece > 500 filtresi uygulanmış) (**c**), hub düğümler ve topluluk yapısı (en bağlantılı düğümler, derece > 100 filtresi uygulanmış) (**d**)

```
# Gephi icin GEXF formatinda kaydet
nx.write_gexf(G, "data/github_network.gexf")
```

9.5. Gerçek Dünya Yorumu

Bu çalışmanın bulguları GitHub açık kaynak ekosistemi hakkında birkaç önemli çıkarım sunmaktadır. Ortalama yol uzunluğunun 3,26 olması, bir geliştiricinin yeni bir teknoloji veya işbirliği fırsatıyla yalnızca birkaç bağlantı aracılığıyla karşılaşabileceğini göstermektedir. Gürbüzlük analizindeki sağlamlık-kırılganlık paradoksu **[1]** ise platformun çok bağlantılı geliştiricilere yapısal bağımlılığını ortaya koymaktadır. Son olarak Topluluk 2'nin ML developer yoğunluğu, teknik uzmanlık alanlarının sosyal ağ yapısına yansıdığını ve bu ağın bilgi yayılımını yönlendiren organik kümeler oluşturduğunu göstermektedir.

Genel olarak elde edilen bulgular, GitHub sosyal ağının klasik rastgele ağ modellerinden ziyade, ölçekten bağımsız ve küçük dünya özellikleri taşıyan karmaşık bir ağ yapısına sahip olduğunu göstermektedir. Bu yapı, açık kaynak ekosisteminde bilginin hızlı yayılmasını sağlarken, aynı zamanda belirli merkezi düğümlere olan bağımlılığı da beraberinde getirmektedir.

BA modelinin GitHub ağına ER modeline kıyasla çok daha iyi uyması tesadüf değildir. GitHub'a yeni katılan bir geliştirici, platformdaki en popüler ve aktif kullanıcıları takip etme eğilimindedir. Bu davranış, tercihli bağlanma (preferential attachment) mekanizmasının birebir gerçek dünya karşılığıdır. Sonuç olarak ağ, BA modelinin temel varsayımını organik biçimde yerine getirmekte ve zaman içinde giderek güçlenen hub düğümleri üretmektedir.

Bu çalışma için kullanılan tüm analizler ve grafikler GitHub deposunda **[9]** mevcuttur.

10. KAYNAKLAR

[1] A. L. Barabási, *Network Science*. Cambridge University Press, 2016. [Online]. Available: http://networksciencebook.com

[2] B. Rozemberczki, C. Allen, and R. Sarkar, "Multi-scale attributed node embedding," *arXiv preprint arXiv:1909.13021*, 2019. [Online]. Available: https://arxiv.org/abs/1909.13021

[3] V. D. Blondel, J. L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.

[4] M. E. J. Newman, "Assortative mixing in networks," *Physical Review Letters*, vol. 89, no. 20, p. 208701, 2002.

[5] P. Erdős and A. Rényi, "On random graphs," *Publicationes Mathematicae*, vol. 6, pp. 290–297, 1959.

[6] R. Xulvi-Brunet and I. M. Sokolov, "Reshuffling scale-free networks: From random to assortative," *Physical Review E*, vol. 70, no. 6, p. 066102, 2004.

[7] A. Hagberg, P. Swart, and D. Schult, "Exploring network structure, dynamics, and function using NetworkX," 2008. [Online]. Available: https://networkx.org

[8] Gephi Consortium, *Gephi: The Open Graph Viz Platform*, Version 0.10, 2009. [Online]. Available: https://gephi.org

[9] E. İnan, "ELE568 GitHub Network Analysis," GitHub, 2026. [Online]. Available: https://github.com/inaneda/ELE568-Network-Science-github-network-analysis